

Cifras Clássicas

1. Cifra de César

1.1 cesar.py — Cifrar e Decifrar

A cifra de César desloca cada letra do alfabeto um número fixo de posições K , definido pela chave. A operação de cifra é dada por:

$$\text{cifra}(c) = (\text{posição}(c) + K) \bmod 26$$

A decifra é simplesmente o processo inverso:

$$\text{decifra}(c) = (\text{posição}(c) - K) \bmod 26$$

O programa `cesar.py` recebe três argumentos: a operação (enc/dec), a chave (letra A-Z) e a mensagem a cifrar/decifrar.

Exemplos de utilização:

```
$ python3 cesar.py enc G "CartagoEstaNoPapo"
IGXZGMUKYZGTUVGVU
$ python3 cesar.py dec G "IGXZGMUKYZGTUVGVU"
CARTAGOESTANOPAPO
```

1.2 cesar_attack.py — Ataque por Força Bruta

A cifra de César é vulnerável a um ataque de força bruta pois apenas existem 26 chaves possíveis (uma por letra do alfabeto). O programa testa todas as 26 chaves, decifrando o criptograma com cada uma e verificando se o resultado contém alguma das palavras fornecidas.

Estratégia do ataque:

- Para cada k de 0 a 25, decifrar o criptograma
- Verificar se alguma das palavras fornecidas aparece no texto decifrado
- Se sim, imprimir a chave e o texto decifrado

Exemplos de utilização:

```
$ python3 cesar_attack.py "IGXZGMUKYZGTUVGVU" BACO TACO
(sem output — nenhuma palavra encontrada)
$ python3 cesar_attack.py "IGXZGMUKYZGTUVGVU" BACO PAPO
G
CARTAGOESTANOPAPO
```

2. Cifra de Vigenère

2.1 vigenere.py — Cifrar e Decifrar

A cifra de Vigenère aplica uma sequência de cifras de César: cada letra do texto é cifrada com a letra correspondente da chave, que se repete ciclicamente. Para uma chave de

comprimento n , a letra na posição i é cifrada com o deslocamento correspondente à letra $\text{chave}[i \bmod n]$.

Vantagem face à César: a mesma letra no texto-limpo pode produzir letras diferentes no criptograma, dificultando a análise de frequência.

Exemplo com chave BACO e texto OLAMUNDO:

Texto	O	L	A
Chave	B (+1)	A (+0)	C (+2)
Cifrado	P	L	C

Exemplos de utilização:

```
$ python3 vigenere.py enc BACO "CifraIndecifavel"
DIHFBIPRFCKTSAXSM
$ python3 vigenere.py dec BACO "DIHFBIPRFCKTSAXSM"
CIFRAINDECIFRAVEL
```

2.2 vigenere_attack.py — Ataque por Análise de Frequência

Atacar a cifra de Vigenère é mais desafiante. Assumindo que o tamanho da chave é conhecido, a estratégia consiste em:

- Dividir o criptograma em fatias — cada fatia contém os caracteres cifrados pela mesma letra da chave
- Cada fatia é essencialmente uma cifra de César — pode ser atacada individualmente
- Para cada fatia, identificar a letra mais frequente (assumindo que corresponde à letra mais comum em Português: A)
- Combinar as chaves parciais de cada fatia para obter a chave completa
- Verificar se o texto decifrado contém alguma das palavras fornecidas

Exemplo de utilização:

```
$ python3 vigenere_attack.py 3
"PGRGARHSFHPRGCVHOJHWEPRZSCJFIVSOFRWUTBKPZGGOPZLHWKPBR" PAPO
PRAIA
POR
ASARMASEOSBAROESASSINALADOSQUEDAOCIDENTALPRAIALUSITANA
```

3. One-Time Pad (OTP)

3.1 otp.py — Implementação Segura

A cifra One-Time Pad (OTP) cifra uma mensagem usando XOR bit a bit com uma chave aleatória do mesmo tamanho. A propriedade fundamental do XOR é que aplicar a operação duas vezes com a mesma chave recupera o texto original:

```
C = M XOR K   (cifrar)
M = C XOR K   (decifrar)
```

O programa `otp.py` suporta três operações:

- `setup n ficheiro` — gera `n` bytes aleatórios seguros (`os.urandom`) para o ficheiro de chave
- `enc ficheiro_msg ficheiro_chave` — cifra a mensagem, guardando em `ficheiro_msg.enc`
- `dec ficheiro_enc ficheiro_chave` — decifra o criptograma, guardando em `ficheiro_enc.dec`

Exemplo de utilização:

```
$ python3 otp.py setup 30 otp.key
$ echo "Mensagem a cifrar" > ptxt.txt
$ python3 otp.py enc ptxt.txt otp.key
$ python3 otp.py dec ptxt.txt.enc otp.key
$ cat ptxt.txt.enc.dec
Mensagem a cifrar
```

3.2 `bad_otp.py` — Implementação com Gerador Fraco

O programa `bad_otp.py` é idêntico ao `otp.py`, com a única diferença de usar um gerador de números pseudo-aleatórios inseguro (`bad_prng`) que usa uma semente de apenas 2 bytes:

```
def bad_prng(n):
    random.seed(random.randbytes(2)) # semente de apenas 2 bytes!
    return random.randbytes(n)
```

Q1 — Diferenças entre `otp.py` e `bad_otp.py`

Do ponto de vista do utilizador, os dois programas comportam-se de forma idêntica — cifram e decifram corretamente sem diferenças visíveis no output. A diferença está na segurança interna:

Característica	<code>otp.py</code>	<code>bad_otp.py</code>
Gerador	<code>os.urandom()</code>	<code>bad_prng()</code>
Sementes possíveis	2^{128} ou mais	apenas 65.536 (2^{16})
Segurança	Criptograficamente seguro	Vulnerável a ataque

A vulnerabilidade é invisível para o utilizador — o criptograma parece igualmente aleatório nos dois casos. Apenas quem conhece a implementação sabe que é fraco.

3.3 `bad_otp_attack.py` — Ataque ao Gerador Fraco

O ataque explora o facto de a semente ter apenas 2 bytes, resultando em apenas 65.536 chaves possíveis. O programa testa todas as sementes possíveis, gerando a chave correspondente e decifrando o criptograma, até encontrar um texto que contenha uma das palavras fornecidas.

Exemplo de utilização:

```
$ python3 bad_otp_attack.py 30 ptxt.txt.enc texto cifrar
Mensagem a cifrar
```

Q2 — O ataque contradiz a segurança absoluta do OTP?

Não contradiz. O teorema da segurança absoluta do OTP (Shannon) assume que a chave é verdadeiramente aleatória, usada apenas uma vez e com tamanho igual ao da mensagem. O `bad_otp.py` não satisfaz a primeira condição — o gerador `bad_prng` não é verdadeiramente aleatório, pois a sua semente tem apenas 2 bytes (65.536 possibilidades).

O ataque explorou a fraqueza do gerador de números aleatórios, e não a cifra OTP em si. Um OTP implementado corretamente com `os.urandom()` continua a ser absolutamente seguro e inquebrável.

Q3 — Reutilização da mesma chave em dois OTPs

Se duas mensagens M1 e M2 forem cifradas com a mesma chave K:

$$C1 = M1 \text{ XOR } K$$
$$C2 = M2 \text{ XOR } K$$

Um atacante que possua ambos os criptogramas pode calcular:

$$C1 \text{ XOR } C2 = (M1 \text{ XOR } K) \text{ XOR } (M2 \text{ XOR } K) = M1 \text{ XOR } M2$$

A chave cancela-se e o atacante obtém o XOR das duas mensagens originais. Conhecendo padrões de linguagem natural (palavras comuns, espaços, pontuação), é relativamente fácil recuperar ambas as mensagens a partir do seu XOR. Este ataque é conhecido como "Two-Time Pad" e demonstra que a reutilização de chaves OTP destrói completamente a segurança.

O ataque é simples de construir em Python:

```
xor_msgs = bytes(a ^ b for a, b in zip(C1, C2))
```

A partir do `xor_msgs`, tentando posições com palavras conhecidas é possível recuperar ambas as mensagens, tornando a regra de utilização única da chave absolutamente fundamental para a segurança do OTP.